Software Requirements Specification

For

CampusMesh: Decentralized Opportunistic Delay-Tolerant Network

For Heterogeneous Campus Communication

Version 1.0

Prepared by

Name	Roll No.	Email
Bhumika Anil Tarmale	2024300259	bhumika.tarmale24@spit.ac.in
Sahil Azad Valvi	2024300266	sahil.valvi24@spit.ac.in
Pranav Solunke	2024300246	pranav.solunke24@spit.ac.in
Instructor:		Prof. Yogita Patil
Course:		Mini Project
Department & Institution:		Sardar Patel Institute of Technology (S.P.I.T.), Mumbai
Date:		21 October 2026

Software Requirements Specification for CampusMesh

Revision History

Version	Primary Author(s)	Description of Version	Date Completed
Draft 1.0	Bhumika Tarmale, Sahil Valvi, Pranav Solunke	Initial full draft of the IEEE 830 SRS document for CampusMesh decentralized delay-tolerant network.	10/21/2026

Software Requirements Specification for CampusMesh *Page 3*

Table of Contents

Revision History 2
Table of Contents 3
1 Introduction 5
1.1 Purpose 5
1.2 Document Conventions 5
1.3 Intended Audience and Reading Suggestions 5
1.4 Product Scope 5
1.5 References 6
2 Overall Description 7
2.1 Product Perspective 7
2.2 Product Functions 7
2.3 User Classes and Characteristics 8
2.4 Operating Environment 8
2.5 Design and Implementation Constraints 9
2.6 User Documentation 10
2.7 Assumptions and Dependencies 10
3 External Interface Requirements 11
3.1 User Interfaces 11
3.2 Hardware Interfaces 12
3.3 Software Interfaces 13
3.4 Communications Interfaces 13
4 System Features 14
4.1 Autonomous Peer Discovery (Wi-Fi mDNS & Subnet UDP Beaconing) 14
4.1.1 Description and Priority 14
4.1.2 Stimulus/Response Sequences 14
4.1.3 Functional Requirements 14
4.2 Opportunistic Bluetooth Low Energy (BLE) Radio Discovery 15
4.2.1 Description and Priority 15
4.2.2 Stimulus/Response Sequences 15
4.2.3 Functional Requirements 15
4.3 Store–Carry–Forward Message Buffer Management 15
4.3.1 Description and Priority 15
4.3.2 Stimulus/Response Sequences 15
4.3.3 Functional Requirements 16
4.4 Dynamic Routing Engine (Epidemic, Spray-and-Wait, Adaptive Utility) 16
4.4.1 Description and Priority 16
4.4.2 Stimulus/Response Sequences 16

4.4.3 Functional Requirements 16
4.5 Multi-Transport Message Dispatch & Socket Streaming (TCP) 17
4.5.1 Description and Priority 17
4.5.2 Stimulus/Response Sequences 17
4.5.3 Functional Requirements 17
4.6 Bluetooth Low Energy GATT MTU Chunking & CRC32 Reassembly 17

Software Requirements Specification for CampusMesh *Page 4*

4.6.1 Description and Priority 17
4.6.2 Stimulus/Response Sequences 17
4.6.3 Functional Requirements 17
4.7 Persistent Event Logging & SQLite Audit Trail 18
4.7.1 Description and Priority 18
4.7.2 Stimulus/Response Sequences 18
4.7.3 Functional Requirements 18
4.8 Embedded HTTP REST API Server & Web Control Plane 19
4.8.1 Description and Priority 19
4.8.2 Stimulus/Response Sequences 19
4.8.3 Functional Requirements 19
4.9 Interactive Real-Time Topology Radar & Network Simulation 19
4.9.1 Description and Priority 19
4.9.2 Stimulus/Response Sequences 19
4.9.3 Functional Requirements 20
4.10 Device Perspective Switching & Autonomous Mobile Node Identification 20
4.10.1 Description and Priority 20
4.10.2 Stimulus/Response Sequences 20
4.10.3 Functional Requirements 21
5 Other Nonfunctional Requirements 22
5.1 Performance Requirements 22
5.2 Safety Requirements 22
5.3 Security Requirements 23
5.4 Software Quality Attributes 23
5.4.1 Usability 23
5.4.2 Reliability 24
5.4.3 Maintainability 24
5.4.4 Portability 24
5.4.5 Efficiency 24
5.5 Business Rules 24
6 Other Requirements 25
Appendix A: Glossary 26
Appendix B: Analysis Models (Use Cases) 27
Appendix C: Data Dictionary 30
Appendix D: To Be Determined (TBD) List 32

Software Requirements Specification for CampusMesh

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) outlines the functional and non-functional requirements for **CampusMesh**, a decentralized, multi-transport, delay-tolerant opportunistic peer-to-peer messaging system engineered for disconnected and intermittently connected campus environments. It details the operational scope, intended user classes, architectural boundaries, external interfaces, algorithmic models, and detailed system features. This document will serve as a formal reference for developers, testers, evaluators, and the course instructor in compliance with the IEEE 830 standard.

1.2 Document Conventions

This document follows the formatting and structural guidelines established by IEEE Std 830-1998:

- **Header:** Font Size: 10 pt, Font Style: Bold Italic, Font: Times New Roman
- **Heading 1:** Font Size: 20 pt, Font Style: Bold, Font: Times New Roman
- **Heading 2 / Sub Heading:** Font Size: 15 pt, Font Style: Bold, Font: Times New Roman
- **Body Content:** Font Size: 12 pt, Font Style: Regular, Font: Times New Roman, Line Spacing: 1.25
- **Requirements Identifiers:** Numbered sequentially as `FR-4.x.y` for functional features and `NFR-5.x.y` for non-functional attributes.

1.3 Intended Audience and Reading Suggestions

This document is primarily intended for:

- **Campus End Users (Students, Faculty, Emergency Staff):** Individuals utilizing the opportunistic mesh for offline messaging during internet blackouts. This section aids in understanding system capabilities and physical mobility reliance.
- **Academic Evaluator / Course Instructor (Prof. Yogita Patil):** To assess whether the distributed systems architecture, algorithmic models, test coverage, and documentation align with IEEE standards and course expectations.
- **Developers and Maintainers:** To clearly understand protocol state machines, buffer capacity constraints, socket serialization, and BLE GATT chunking procedures.
- **Network Testers:** To design automated test cases validating message deduplication, token splitting, and multi-hop routing benchmarks.

This document is organized following the IEEE 830 structure. Readers are encouraged to review Sections 1 and 2 for conceptual grounding before diving into the formal technical requirements in Sections 3 through 6 and Appendices.

1.4 Product Scope

CampusMesh provides decentralized, infrastructure-free communication across university campuses by implementing the **Store–Carry–Forward** Delay-Tolerant Networking (DTN) model. Users can compose, buffer, and exchange priority-rated messages across heterogeneous devices (Windows,

Ubuntu, Android, iOS) without requiring an active internet connection, cellular service, or centralized routing servers. By operating over both high-speed Wi-Fi sockets and short-range Bluetooth Low Energy (BLE) GATT channels, the system ensures that physical mobility bridges partitioned networks.

Software Requirements Specification for CampusMesh *Page 6*

The platform features an autonomous peer discovery daemon, thread-safe memory-bounded local storage (10 MB cap), a dynamic routing engine supporting Epidemic, Spray-and-Wait, and Adaptive Multi-Metric Utility routing, length-prefixed TCP streaming, MTU-aware BLE packet slicing with CRC32 integrity verification, SQLite relational event logging, and an embedded zero-dependency HTTP REST control center featuring an interactive HTML5 canvas radar topology.

1.5 References

1. V. Cerf, S. Burleigh, A. Hooke, L. Torgerson, R. Durst, K. Scott, K. Fall, and H. Weiss, “Delay-Tolerant Networking Architecture,” *RFC 4838*, Internet Engineering Task Force (IETF), Apr. 2007. https://datatracker.ietf.org/doc/html/rfc4838
2. K. Fall, “A Delay-Tolerant Network Architecture for Challenged Internets,” in *Proc. ACM SIGCOMM 2003 Conference (SIGCOMM ’03)*, Karlsruhe, Germany, 2003, pp. 27–34.
3. A. Vahdat and D. Becker, “Epidemic Routing for Partially-Connected Ad Hoc Networks,” Technical Report CS-200006, Duke University, Tech. Rep., Jul. 2000.
4. T. Spyropoulos, K. Psounis, and C. S. Raghavendra, “Spray and Wait: An Efficient Routing Scheme for Intermittently Connected Mobile Networks,” in *Proc. ACM SIGCOMM Workshop on Delay-Tolerant Networking (WDTN ’05)*, Philadelphia, PA, USA, 2005, pp. 252–259.
5. Bluetooth Special Interest Group (SIG), “Bluetooth Core Specification Version 5.3: Low Energy Generic Attribute Profile (GATT),” Kirkland, WA, USA, Jul. 2021.
6. IEEE Computer Society, “IEEE Recommended Practice for Software Requirements Specifications,” *IEEE Std 830-1998*, Oct. 1998.

Software Requirements Specification for CampusMesh *Page 7*

2 Overall Description

2.1 Product Perspective

In modern academic campus environments, everyday communications rely entirely on centralized infrastructure: cellular base stations, campus Wi-Fi access points, core switches, and cloud servers. During severe power failures, monsoon storms, network hardware breakdowns, or high-density campus events, centralized networks frequently collapse, creating total information dead zones. Traditional mobile messaging applications (WhatsApp, Telegram, Slack) fail completely without an active internet gateway.

CampusMesh is introduced to establish resilient, decentralized connectivity in such environments. Operating under the Delay-Tolerant Networking (DTN) store-carry-forward model, each device running CampusMesh functions as a standalone packet router and carrier. When two students or faculty members walk within radio range, their devices autonomously establish direct physical links over local Wi-Fi or Bluetooth Low Energy, exchanging stored messages before parting ways. Over time, messages cascade across the campus from student to student until reaching the target destination or diffusing across the entire community.

2.2 Product Functions

- Autonomous zero-configuration discovery of neighboring peers via Wi-Fi mDNS, UDP subnet multicasting (`239.255.77.88:5354`), and BLE advertising.
- Dynamic radio encounter detection and contact lifecycle tracking via `ContactManager`.
- Store-Carry-Forward message buffering with strict 10 MB memory capacity enforcement.
- Binary duplicate message detection and idempotent packet rejection.
- Real-time timestamp-based Time-To-Live (TTL) tracking with automatic expiration purging.
- Pluggable routing strategy selection: Epidemic Flooding, Spray-and-Wait binary token splitting, and Adaptive Utility scoring.
- Multi-metric adaptive utility ranking balancing Priority (P), TTL Urgency (T), and Buffer Pressure (B).
- High-speed stream-safe 4-byte big-endian length-prefixed TCP socket transport.
- Bluetooth Low Energy MTU-aware slicing (23–512 bytes) with 13-byte headers and end-to-end CRC32 checksum reassembly.
- Thread-safe relational event auditing to local SQLite database (`database/campusmesh.db`).
- Embedded zero-dependency HTTP server delivering a real-time web control center on port 8080.
- Interactive HTML5 Canvas radar displaying live node topology, multi-transport links, and animated packet hops.
- Autonomous mobile device identification and independent client perspective switching.

Software Requirements Specification for CampusMesh *Page 8*

2.3 User Classes and Characteristics

The system serves four distinct user classes:

- **Campus Users (Students, Faculty, Staff):**
  - Non-technical or moderately technical users.
  - Operate smartphones and laptops while traversing campus buildings.
  - Use the web dashboard to compose local alerts, view incoming notices, and track buffer contents.
- **Emergency Response & Security Coordinators:**
  - Responsible for safety broadcasts and critical administrative alerts.
  - Require high-priority message preemption and broad mesh dissemination without internet access.
- **Network Researchers and Lab Evaluators:**
  - Evaluate routing performance, buffer pressure distributions, and packet delivery ratios.
  - Tune adaptive utility weights (w_P, w_T, w_B), trigger multi-hop simulations, and analyze SQLite event logs.
- **Academic Course Evaluator (Prof. Yogita Patil):**
  - Assesses software engineering compliance, architectural separation, and automated test suite execution (43/43 tests).

2.4 Operating Environment

Minimum system requirements include:

- **RAM:** 512 MB minimum (10 MB dedicated message buffer cap).
- **Disk Space:** 50 MB free local disk space for SQLite database and binaries.
- **Network Interfaces:** 802.11 b/g/n/ac Wi-Fi Network Adapter and/or Bluetooth 4.2 / 5.0+ LE Controller.
- **Java Runtime Environment:** OpenJDK / Oracle JDK 8 or higher (tested on Java 23).
- **Operating Systems:** Microsoft Windows 10/11, Ubuntu Linux 20.04/22.04 LTS, macOS, Android 10+, iOS 14+.
- **Web Browser:** Google Chrome 80+, Mozilla Firefox 75+, Apple Safari 13+, or Microsoft Edge 80+.

2.5 Design and Implementation Constraints

1. **Zero External Web Frameworks:** The backend control plane must execute using standard Java SE `com.sun.net.httpserver.HttpServer` with zero third-party dependencies (no Spring Boot, Tomcat, or Node.js runtime).
2. **Architectural Separation:** The routing engine must decide *what* messages to forward, while transport sockets decide *how* messages are physically transmitted.
3. **Memory Capping:** The message store is strictly bounded at 10 MB ($10,485,760$ bytes) to guarantee execution stability on low-resource mobile nodes.
4. **Intermittent Link Tolerance:** Physical radios may disconnect at any moment during transit; all socket operations must timeout safely without corrupting buffered data.
5. **BLE MTU Slicing:** Bluetooth Low Energy links must handle default 23-byte MTU boundaries up to 512-byte negotiated frames without losing chunks or overflowing buffers.
6. **Stateless REST Communication:** The web dashboard must poll state via REST JSON endpoints without requiring persistent session cookies or client-side authentication.

2.6 User Documentation

The project provides comprehensive user and developer documentation:

- `README.md`: Comprehensive setup guide, architecture breakdown, and compilation instructions.
- Embedded Web Dashboard Tooltips: Interactive guidance on adaptive weight sliders, buffer metrics, and radio legends.
- Interactive Terminal CLI (`interactive.bat`): Built-in help menus and syntax instructions.
- Automated Test Diagnostics: Real-time console test logs verifying 43 unit and functional test assertions.

2.7 Assumptions and Dependencies

- It is assumed that devices physically move across campus, generating opportunistic radio encounters within 10–50 meters.
- It is assumed that devices have either Wi-Fi or Bluetooth radios enabled.
- It is assumed that device real-time clocks are reasonably synchronized for valid message TTL comparisons.
- It is assumed that local file system write permissions exist for the SQLite database at `database/campusmesh.db`.

3 External Interface Requirements

3.1 User Interfaces

The system provides two primary user interfaces:

- **Web Control Dashboard (Port 8080):**
  - *Top Header & Perspective Switcher:* Dropdown to toggle operating views (`LAPTOP_WIN`, `PHONE_MOBILE`, `PHONE_AND`, etc.), strategy indicator, and buffer gauge.
  - *HTML5 Canvas Topology Radar:* Displays live nodes, animated sweep beams, Wi-Fi/BLE link lines, and traveling packet animations.
  - *Decision Matrix Panel:* Interactive sliders for w_P, w_T, w_B with live utility ranking tables.
  - *Message Composer:* Input controls for Sender, Destination, Priority (High/Med/Low), TTL slider, Copy Tokens (L), and payload text.
  - *Store-Carry-Forward Buffer:* Interactive cards with real-time TTL progress bars.
  - *BLE Frame Inspector:* Visual breakdown of 13-byte headers, chunks, and CRC32 hex values.
  - *SQLite Audit Stream:* Real-time feed of the last 15 logged system events.
- **Interactive Terminal Console (CLI):** Terminal-based ANSI menu providing direct command-line execution and simulation control.

3.2 Hardware Interfaces

- Standard 802.11 b/g/n/ac Wi-Fi Network Interface Cards (NICs).
- Bluetooth Low Energy 4.2 / 5.0+ Host Controller Interfaces (HCI).
- Solid-state local storage for persistent SQLite database logs.

3.3 Software Interfaces

- Java Virtual Machine (JVM): Java SE 8 or higher.
- SQLite Relational Database: Accessed via `sqlite-jdbc-3.36.0.3.jar`.
- Embedded HTTP Server: `com.sun.net.httpserver.HttpServer`.
- Web Browsers: HTML5 Canvas 2D API, Fetch API, and LocalStorage API.

3.4 Communications Interfaces

- **Wi-Fi TCP Streaming:** 4-byte big-endian length prefix followed by UTF-8 JSON payload.
- **Wi-Fi Subnet Multicast:** UDP beacons on `239.255.77.88:5354` every 5 seconds.
- **Bluetooth LE GATT:** Service UUID `0000FE60`, 13-byte self-describing binary headers, CRC32 error detection.
- **REST HTTP API:** JSON endpoints on port 8080 supporting Cross-Origin Resource Sharing (CORS).

4 System Features

4.1 Autonomous Peer Discovery (Wi-Fi mDNS & Subnet UDP Beaconing)

4.1.1 Description and Priority

Enables zero-configuration proximity discovery across local Wi-Fi networks without requiring central directory infrastructure. **Priority: High.**

4.1.2 Stimulus/Response Sequences

1. Node initializes `DiscoveryManager` on startup.
2. `WifiDiscoveryService` binds to multicast address `239.255.77.88:5354`.
3. Background thread broadcasts JSON discovery beacons every 5 seconds.
4. Neighboring nodes parse beacon metadata and update `PeerRegistry`.
5. Nodes not heard from within 15 seconds are purged as stale.

4.1.3 Functional Requirements

- **FR-4.1.1:** The system shall broadcast JSON UDP discovery beacons to `239.255.77.88:5354` every 5000 ms.
- **FR-4.1.2:** The system shall parse inbound beacons and register peer ID, platform, IP address, port, and transports in `PeerRegistry`.
- **FR-4.1.3:** The system shall evict peer entries whose last seen timestamp exceeds 15000 ms.

4.2 Opportunistic Bluetooth Low Energy (BLE) Radio Discovery

4.2.1 Description and Priority

Discovers adjacent physical nodes in complete Wi-Fi dead zones via BLE advertisements. **Priority: High.**

4.2.2 Stimulus/Response Sequences

1. Node activates BLE peripheral service advertising UUID `0000FE60`.
2. Scanning nodes detect service and connect to GATT server.
3. Scanning node reads Characteristic `0000FE61` to obtain peer identity.
4. Peer is merged into `PeerRegistry` with transport capability `BLE`.

4.2.3 Functional Requirements

- **FR-4.2.1:** The system shall broadcast BLE advertisements specifying Service UUID `0000FE60-0000-1000-8000-00805F9B34FB`.
- **FR-4.2.2:** The system shall query Characteristic `0000FE61` to ingest remote peer metadata.
- **FR-4.2.3:** The system shall merge multi-transport capabilities into a unified peer record.

Software Requirements Specification for CampusMesh

4.3 Store–Carry–Forward Message Buffer Management

4.3.1 Description and Priority

Maintains persistent in-memory message storage while nodes transit between network partitions. **Priority: High.**

4.3.2 Stimulus/Response Sequences

1. Message arrives via local creation or socket reception.
2. Buffer verifies that `messageId` is not a duplicate.
3. Buffer verifies remaining TTL is positive.
4. Buffer checks capacity (10 MB cap) and commits message.
5. Querying the buffer triggers automatic purging of expired messages.

4.3.3 Functional Requirements

- **FR-4.3.1:** The system shall strictly enforce a 10 MB ($10,485,760$ bytes) upper storage boundary.
- **FR-4.3.2:** The system shall reject duplicate messages possessing an identical `messageId`.
- **FR-4.3.3:** The system shall calculate $\text{remainingTTL} = \text{expiresAt} - \text{now}$ and purge expired entries automatically.

4.4 Dynamic Routing Engine (Epidemic, Spray-and-Wait, Adaptive Utility)

4.4.1 Description and Priority

Selects which messages to forward across an active encounter based on the active strategy. **Priority: High.**

4.4.2 Stimulus/Response Sequences

1. Encounter begins with remote peer P.
2. Routing engine queries active strategy with candidate messages.
3. Under Epidemic, all unreceived messages are forwarded.
4. Under Spray-and-Wait, copy tokens are halved until $L=1$.
5. Under Adaptive, candidates are ranked by multi-metric utility score $U(M)$.

4.4.3 Functional Requirements

- **FR-4.4.1:** The system shall support runtime switching between Epidemic, Spray-and-Wait, and Adaptive routing.
- **FR-4.4.2:** In Spray-and-Wait, messages with $L > 1$ tokens shall split tokens binary ($\lfloor L/2 \rfloor$ sent, $\lceil L/2 \rceil$ kept); messages with $L = 1$ shall forward only to final destination.
- **FR-4.4.3:** In Adaptive routing, utility shall be calculated as $U(M) = w_P \cdot P(M) + w_T \cdot T(M) + w_B \cdot B(M)$, where $w_P + w_T + w_B = 1.00$.

Software Requirements Specification for CampusMesh

4.5 Multi-Transport Message Dispatch & Socket Streaming (TCP)

4.5.1 Description and Priority

Transfers DTN messages over physical Wi-Fi TCP sockets using stream-safe framing. **Priority: High.**

4.5.2 Stimulus/Response Sequences

1. Routing engine designates message for dispatch.
2. Socket connects to remote peer IP and port.
3. System prepends a 4-byte big-endian length prefix.
4. System writes serialized UTF-8 JSON payload bytes.
5. Remote receiver reads exact length via `readFully()`.

4.5.3 Functional Requirements

- **FR-4.5.1:** The system shall frame all TCP payloads with a 4-byte big-endian integer length header.
- **FR-4.5.2:** Receiving sockets shall ingest the exact stream length indicated by the header.

4.6 Bluetooth Low Energy GATT MTU Chunking & CRC32 Reassembly

4.6.1 Description and Priority

Slices messages into MTU-compliant BLE GATT packets and reassembles them with CRC32 verification. **Priority: High.**

4.6.2 Stimulus/Response Sequences

1. Payload is partitioned into chunk payloads of size $\text{MTU} - 13$.
2. 13-byte headers containing Transfer ID, Chunk Index, and CRC32 are prepended.
3. Chunks are transmitted sequentially over Characteristic `0000FE62`.
4. Receiver reassembles chunks and verifies CRC32 checksum before buffer ingestion.

4.6.3 Functional Requirements

- **FR-4.6.1:** The system shall attach a 13-byte header to every BLE chunk (magic bytes `0x43 0x4D`, version `0x02`, TxID, Index, Total, CRC32).
- **FR-4.6.2:** The receiver shall verify end-to-end CRC32 checksums, rejecting corrupted transmissions.

Software Requirements Specification for CampusMesh

4.7 Persistent Event Logging & SQLite Audit Trail

4.7.1 Description and Priority

Maintains a queryable relational audit log of all system transitions and routing actions. **Priority: Medium.**

4.7.2 Stimulus/Response Sequences

1. System event occurs (e.g., `MESSAGE_FORWARDED`).
2. Event details are dispatched to `EventLogger`.
3. Thread-safe SQLite connection writes record into table `events`.
4. Record is queryable via REST endpoint `GET /api/events`.

4.7.3 Functional Requirements

- **FR-4.7.1:** The system shall persist audit events in local SQLite file `database/campusmesh.db`.
- **FR-4.7.2:** Table `events` shall record timestamp, event type, peer ID, message ID, remote peer ID, and metadata JSON.

4.8 Embedded HTTP REST API Server & Web Control Plane

4.8.1 Description and Priority

Serves the web dashboard and REST API endpoints using standard JDK libraries on port 8080. **Priority: High.**

4.8.2 Stimulus/Response Sequences

1. Server binds to `0.0.0.0:8080`.
2. Web browser requests dashboard assets (HTML/CSS/JS).
3. Browser polling loop fetches `/api/status`, `/api/peers`, and `/api/buffer` every 1.5 seconds.

4.8.3 Functional Requirements

- **FR-4.8.1:** The server shall operate without external web dependencies using `com.sun.net.httpserver.HttpServer`.
- **FR-4.8.2:** The server shall provide REST JSON endpoints for status, peers, buffer, routing strategy, and event inspection.

Software Requirements Specification for CampusMesh

4.9 Interactive Real-Time Topology Radar & Network Simulation

4.9.1 Description and Priority

Renders an interactive HTML5 canvas radar displaying live mesh topology, animated link beams, and packet hops. **Priority: Medium.**

4.9.2 Stimulus/Response Sequences

1. Dashboard initializes canvas radar rendering at 60 FPS.
2. Nodes and links (Wi-Fi dashed cyan, BLE dashed purple) are rendered radially.
3. User clicks "Run Multi-Hop Demo"; backend executes 4-node simulation.
4. Animated blue packets travel between nodes across the canvas.

4.9.3 Functional Requirements

- **FR-4.9.1:** The canvas radar shall render sweeping beams and dynamic links at 60 FPS.
- **FR-4.9.2:** Clicking any node on the canvas shall automatically set it as the message composer destination.

4.10 Device Perspective Switching & Autonomous Mobile Node Identification

4.10.1 Description and Priority

Allows any connected device to operate under its own identity, auto-detecting mobile devices over Wi-Fi. **Priority: High.**

4.10.2 Stimulus/Response Sequences

1. Mobile phone opens `http://<laptop-ip>:8080`.
2. Backend identifies remote client IP and auto-registers `PHONE_MOBILE` in `PeerRegistry`.
3. Frontend detects mobile User-Agent / remote flag and sets operating perspective to `PHONE_MOBILE`.
4. Radar canvas displays `PHONE_MOBILE (YOU)` glowing in vibrant green.
5. Composer pre-populates `Sender: PHONE_MOBILE` and `Destination: LAPTOP_WIN`.

4.10.3 Functional Requirements

- **FR-4.10.1:** The backend shall detect remote client IP addresses and auto-register `PHONE_MOBILE` as a discovered peer.
- **FR-4.10.2:** The frontend shall allow switching active perspectives via the top navbar dropdown, persisting preference in `localStorage`.
- **FR-4.10.3:** The radar canvas shall dynamically render the glowing green local node indicator on the active perspective node.

Software Requirements Specification for CampusMesh

5 Other Nonfunctional Requirements

5.1 Performance Requirements

- **Buffer Latency:** Message buffer queries, insertions, and duplicate checks shall complete within 5 ms for up to 1,000 entries.
- **Adaptive Calculation Speed:** Ranking 100 candidate messages in the decision matrix shall complete in under 2 ms.
- **Socket Throughput:** TCP length-prefixed streaming over Wi-Fi shall achieve up to 10 MB/s transfer speeds.
- **BLE Slicing Latency:** Chunking a 10 KB payload with CRC32 computation shall take less than 20 ms.

5.2 Safety Requirements

- **Memory Safety:** Strict 10 MB buffer capacity prevents JVM heap exhaustion.
- **Loop Prevention:** Spray-and-Wait token splitting guarantees finite replication, avoiding infinite broadcast loops.

5.3 Security Requirements

- **Payload Integrity:** CRC32 checksums verify data integrity across lossy radio links.
- **Stream Protection:** 4-byte length prefixes prevent TCP boundary injection attacks.
- **Zero Telemetry:** No user data or telemetry is leaked to external internet entities.

5.4 Software Quality Attributes

- **Usability:** Intuitive web dashboard requiring no technical terminal knowledge.
- **Reliability:** Graceful socket timeout recovery; unsent messages remain buffered for subsequent encounters.
- **Maintainability:** Clean modular architecture separating routing, transport, buffer, discovery, and logging.
- **Portability:** Identical execution across Windows, Linux, Android, and iOS.

5.5 Business Rules

- **BR-1:** Spray-and-Wait messages with $L=1$ token shall only be delivered to their final destination node.
- **BR-2:** Expired messages ($\text{remainingTTL} \le 0$) shall never be forwarded or accepted.
- **BR-3:** Adaptive routing weights must strictly sum to unity ($w_P + w_T + w_B = 1.00$).

6 Other Requirements

The system provides SQLite audit log export to JSON/CSV, a buffer reset option, WCAG 2.1 Level AA color contrast compliance, and full academic satisfaction for the SPIT Mini Project curriculum.

Software Requirements Specification for CampusMesh

Appendix A: Glossary

Sr. No.	Term	Description
1	**DTN**	Delay-Tolerant Networking: An architecture for communication across challenged and intermittently connected networks.
2	**Store–Carry–Forward**	The routing paradigm where nodes physically carry stored message bundles between disconnected partitions.
3	**P2P**	Peer-to-Peer: Decentralized architecture where all nodes hold symmetric routing privileges.
4	**mDNS**	Multicast DNS: Zero-configuration protocol for discovering hostnames and services on local subnets.
5	**BLE**	Bluetooth Low Energy: Low-power wireless radio technology for short-range ad-hoc communication.
6	**GATT**	Generic Attribute Profile: BLE specification defining Service and Characteristic data structures.
7	**MTU**	Maximum Transmission Unit: Maximum physical packet size allowable across a link (23 to 512 bytes).
8	**CRC32**	Cyclic Redundancy Check 32-bit: Polynomial checksum verifying packet integrity over wireless channels.
9	**TTL**	Time-To-Live: Duration in seconds before a message bundle expires and is evicted from memory.
10	**Epidemic**	Flooding DTN algorithm replicating messages to every newly encountered peer.
11	**Spray-and-Wait**	Controlled replication protocol spraying L copies via binary token splitting before waiting for destination.
12	**Adaptive**	Utility-based routing ranking candidate messages by Priority (P), Urgency (T), and Buffer Pressure (B).
13	**REST**	Representational State Transfer: Architectural style utilizing HTTP methods and JSON data exchange.
14	**SRS**	Software Requirements Specification: Formal technical document outlining software behavior.
15	**SPIT**	Sardar Patel Institute of Technology: Autonomous engineering college in Andheri (W), Mumbai.

Software Requirements Specification for CampusMesh

Appendix B: Analysis Models

The use cases below serve as the formal analysis model for the system, directly cross-referenced from Section 4.

Use Case 1: Autonomous Peer Discovery via Wi-Fi Beacon

Actor: Participating Campus Node

Description: Node discovers adjacent peers across the local Wi-Fi subnet without manual configuration.

Steps: (1) Node starts `WifiDiscoveryService`; (2) Joins multicast group `239.255.77.88:5354`; (3) Emits periodic UDP beacons; (4) Remote peers record node in `PeerRegistry`; (5) Active encounter is notified to `ContactManager`.

Use Case 2: BLE Service Discovery and Handshake

Actor: BLE-Enabled Mobile / Laptop Node

Description: Node detects nearby devices using Bluetooth Low Energy advertising and GATT characteristics.

Steps: (1) Advertises Service UUID `0000FE60`; (2) Remote scanner connects to GATT server; (3) Reads Characteristic `0000FE61` to ingest identity; (4) Peer is registered with transport `BLE` in local registry.

Use Case 3: Enqueue and Store Message in Local Buffer

Actor: Campus User / System Node

Description: A new message is created and committed to the bounded local DTN buffer.

Steps: (1) User composes message; (2) Buffer verifies message is not duplicate; (3) Verifies TTL is valid; (4) Confirms capacity $\le 10\text{ MB}$; (5) Commits entry and logs `MESSAGE_CREATED` to SQLite.

Use Case 4: Evaluate Candidates via Adaptive Routing Matrix

Actor: Routing Engine

Description: Evaluates all buffered messages to rank forwarding candidates during an encounter.

Steps: (1) Contact encounter begins; (2) Engine queries `AdaptiveRouter`; (3) Computes P, T, B and utility $U(M) = w_P \cdot P + w_T \cdot T + w_B \cdot B$; (4) Sorts candidates descending by utility; (5) Dispatches top candidates.

Use Case 5: Deliver Message via Length-Prefixed TCP Stream

Actor: Transport Layer

Description: Transfers an outbound message to a discovered peer over a direct Wi-Fi TCP socket.

Steps: (1) Opens TCP socket to remote peer; (2) Writes 4-byte big-endian integer length; (3) Streams JSON payload bytes; (4) Receiver ingests payload with `readFully()`; (5) Logs `MESSAGE_FORWARDED` to SQLite.

Use Case 6: Chunk and Reassemble BLE GATT Transmission

Actor: BLE Transport Layer

Description: Slices large DTN messages into MTU chunks and reassembles them with CRC32 verification.

Steps: (1) Slices payload into chunks matching MTU; (2) Attaches 13-byte headers and CRC32; (3) Sends chunks over Characteristic `0000FE62`; (4) Receiver verifies CRC32; (5) Commits payload to buffer.

Software Requirements Specification for CampusMesh

Use Case 7: Audit Event Logging to SQLite Database

Actor: System Logging Subsystem

Description: Records node lifecycle, encounters, and routing decisions into relational SQLite tables.

Steps: (1) Event is triggered; (2) Parameters sent to `EventLogger`; (3) Transaction written to table `events`; (4) Event stream appears in web dashboard.

Use Case 8: Inspect and Control Node via Web Dashboard

Actor: Campus User / Researcher

Description: User monitors node metrics, buffer status, and radar links via web browser.

Steps: (1) Opens `http://localhost:8080`; (2) Server returns dashboard assets; (3) Radar renders live links; (4) User adjusts adaptive sliders; (5) Server updates weights in real time.

Use Case 9: Execute Automated Multi-Hop Mesh Simulation

Actor: Network Researcher / Evaluator

Description: Runs an automated simulation demonstrating multi-hop message forwarding.

Steps: (1) Triggers simulation button; (2) Backend executes hops across 4 nodes; (3) Blue animated packets traverse radar; (4) Multi-hop events recorded in SQLite.

Use Case 10: Auto-Detect Mobile Client and Invert Perspective

Actor: Mobile Phone User

Description: Mobile browser connects to laptop server and operates under its own identity.

Steps: (1) Mobile opens dashboard via Wi-Fi IP; (2) Server registers `PHONE_MOBILE`; (3) Radar highlights `PHONE_MOBILE (YOU)`; (4) Composer defaults to `Sender: PHONE_MOBILE`, `Destination: LAPTOP_WIN`; (5) Message sent from phone buffers correctly.

Use Case 11: Evaluate SRS and System Test Compliance

Actor: Academic Course Evaluator (Prof. Yogita Patil)

Description: Evaluator verifies software execution, test suite passes, and requirement compliance.

Steps: (1) Runs `build.bat`; (2) Executes test suites (43/43 tests); (3) Verifies 100% pass rate; (4) Audits SRS document for academic grading.

Appendix C: Data Dictionary

Name	Type	Category	Possible Values	Description
`messageId`	String	State Variable	e.g., "M001", "M1_4920"	Globally unique identifier for each message bundle.
`sourceId`	String	State Variable	"LAPTOP_WIN", "PHONE_MOBILE"	Peer identifier of the originating sender node.
`destinationId`	String	State Variable	"LAPTOP_UBU", "BROADCAST"	Intended destination node ID or broadcast identifier.
`priority`	Enum	State Variable	LOW, MEDIUM, HIGH	Priority classification determining routing factor $P(M)$.
`priorityWeight`	Double	Derived Value	0.33, 0.67, 1.00	Numeric priority weight used in adaptive utility scoring.
`payload`	String	Input / State	UTF-8 text string	Textual content transported across the mesh.
`createdAt`	Instant	State Variable	ISO-8601 UTC	Timestamp when message was initially generated.
`expiresAt`	Instant	State Variable	ISO-8601 UTC	Timestamp when message expires and becomes invalid.
`ttlSeconds`	Long	Parameter	60 to 86400 seconds	Total initial lifespan duration assigned to message.
`remainingTTL`	Long	Derived Value	≥ 0 seconds	Live remaining validity seconds before eviction.
`copyTokens`	Integer	State Variable	1 to 16 (default: 4)	Spray-and-Wait replication tokens remaining (L).
`peerId`	String	State Variable	"LAPTOP_WIN", "PHONE_AND"	Unique identifier representing a participating node.
`platform`	String	Metadata	WINDOWS, ANDROID, IOS, UBUNTU	Operating system environment of the node.
`ipAddress`	String	Network State	Valid IPv4/IPv6 address	Network IP address for Wi-Fi TCP socket connections.
`port`	Integer	Network State	1024 to 65535	TCP port number for sockets and HTTP dashboard.
`wP`	Double	Parameter	0.00 to 1.00 (default: 0.40)	Adaptive weight assigned to Priority Factor.
`wT`	Double	Parameter	0.00 to 1.00 (default: 0.40)	Adaptive weight assigned to TTL Urgency Factor.

Name	Type	Category	Possible Values	Description
`wB`	Double	Parameter	0.00 to 1.00 (default: 0.20)	Adaptive weight assigned to Buffer Pressure Factor.
`utility`	Double	Derived Value	0.00 to 1.00	Overall forwarding score $U(M) = w_P P + w_T T + w_B B$.
`bufferCapacity`	Long	Constant	10,485,760 bytes (10 MB)	Strict upper memory cap for local storage.
`bufferUsedBytes`	Long	State Variable	0 to 10,485,760 bytes	Total bytes currently occupied in message buffer.
`crc32Checksum`	Long	Checksum	32-bit unsigned integer	CRC32 polynomial checksum verifying BLE chunk integrity.
`activeNodeId`	String	UI State	"LAPTOP_WIN", "PHONE_MOBILE"	Current operating identity active on this device.

Appendix D: To Be Determined (TBD) List

No TBD (To Be Determined) items are currently identified in this document. All architectural parameters, radio interface specifications, routing equations, external protocols, and verification criteria have been fully specified and validated as of Version 1.0. This appendix will be updated with a numbered list of any TBD markers introduced in future revisions.